

17

Applications of Stable Diffusion

In the previous chapter, we learned about how diffusion models work, the architecture of Stable Diffusion, and diffusers – the library.

While we learned about generating images, unconditional and conditional (from a text prompt), we still did not learn about having the ability to control the images – for example, I might want to replace a cat in an image with a dog, make a person stand in a certain pose, or replace the face of a superhero with a subject of interest. In this chapter, we will learn about the model training process and coding some of the applications of diffusion that help in achieving the above. In particular, we will cover the following topics:

- In-painting to replace objects within an image from a text prompt
- Using ControlNet to generate images in a specific pose from a text prompt
- Using DepthNet to generate images using a depth-of-reference image and text prompt
- Using SDXL Turbo to generate images faster from a text prompt
- Using Text2Video to generate video from a text prompt

The code used in this chapter is available in the `Chapter17` folder in the GitHub repo at <https://bit.ly/mcvp-2e>. You can run the code from the notebooks and leverage them to understand all the steps.



As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the `supplementary_sections` folder within each chapter's directory for new and useful content.

In-painting

In-painting is the task of replacing a certain portion of an image with another image. An example of in-painting is as follows:

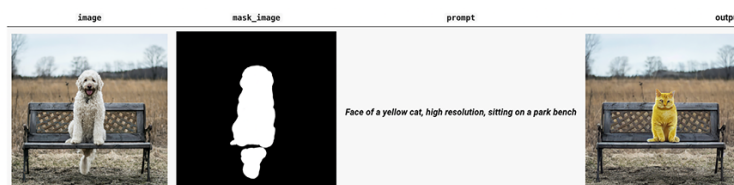


Figure 17.1: The first three items—*image*, *mask_image*, and *prompt*—serve as the inputs, while the rightmost image represents the output of the in-painting process.

In the preceding image, we provide the mask corresponding to the subject that we want to replace – a dog. Additionally, we provide the prompt that we want to use to generate an image. Using the mask and prompt, we should generate an output that satisfies the prompt while keeping the rest of the image intact.

An in the following section, we will understand the model training workflow of in-painting.

Model training workflow

In-painting model is trained as follows:

1. The input requires an image and a caption associated with the input.
2. Pick a subject (a dog in *Figure 17.1*) that is mentioned in the caption and obtain a mask corresponding to the subject.
3. Use the caption as a prompt.
4. Pass the original image through a variational auto-encoder that down-scales the input image (let's say from a 512x512 image to a 64x64 image) to extract the latents corresponding to the original image.
5. Create text latents (that is, embeddings, using OpenAI CLIP or any other embeddings model) corresponding to the prompt. Pass the text embeddings and noise as input to train a U-Net model that outputs the latents.
6. Fetch the original latents (obtained in *step 4*), resized mask (obtained in *step 2*), and latents (obtained in *step 5*) to segregate the background latents and the latents corresponding to the mask region. In essence, the latents in this step are calculated as $\text{original_image_latents} * (1 - \text{mask}) + \text{text_based_latents} * \text{mask}$.
7. Once all the timesteps are finished, we obtain the latents that correspond to the prompt.
8. These latents are passed through a **variational autoencoder (VAE)** decoder to get the final image. The VAE ensures harmony within the generated image.

The overall workflow of in-painting is as follows:

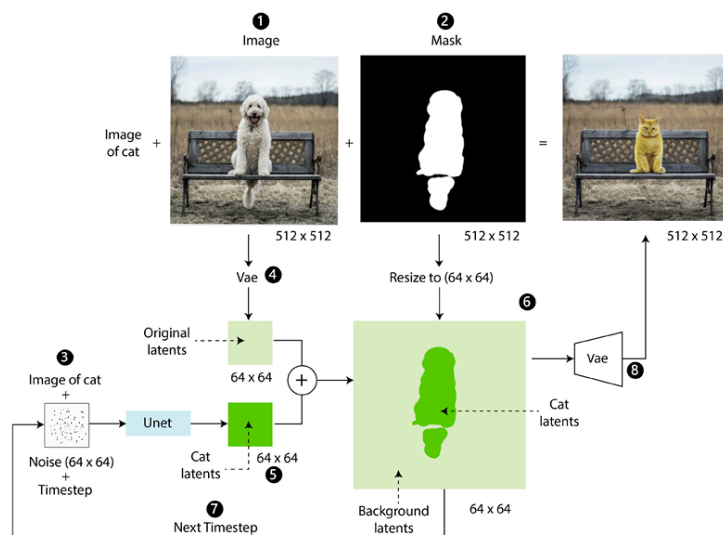


Figure 17.2: Workflow of in-painting

Now that we understand the workflow, let us go ahead and learn about using Stable Diffusion to perform in-painting in the next section.

In-painting using Stable Diffusion

To perform in-painting on an image, we will use the `diffusers` package and the Stable Diffusion pipeline within it. Let us code up in-painting as follows:



The following code is available in the `image_inpainting.ipynb` file in the `Chapter17` folder in the GitHub repository at <https://bit.ly/mcvp-2e>

1. Install the required packages:

```
!pip install diffusers transformers accelerate
```

2. Import the required libraries:

```
import PIL
import requests
import torch
from io import BytesIO
from diffusers import StableDiffusionInpaintPipeline
```

3. Define the pipeline for in-painting:

```
pipeline = StableDiffusionInpaintPipeline.from_pretrained(
    "runwayml/stable-diffusion-inpainting",
    torch_dtype=torch.float16)
pipeline = pipeline.to("cuda")
```

In the preceding code, we leverage the in-painting model developed by `runwayml`. Further, we specify that all the weights have a precision of float16 and not float32 to reduce the memory footprint.

4. Get the image and its corresponding mask from the corresponding URLs:

```
def download_image(url):
    response = requests.get(url)
    return PIL.Image.open(BytesIO(response.content)).convert("RGB")
img_url = "https://raw.githubusercontent.com/CompVis/latent-diffusion/main/data/inpainting_exar
mask_url = "https://raw.githubusercontent.com/CompVis/latent-diffusion/main/data/inpainting_exi
init_image = download_image(img_url).resize((512, 512))
mask_image = download_image(mask_url).resize((512, 512))
```

The original image and the corresponding mask are as follows:



Figure 17.3: The image and the mask of the object you want to replace

You can use standard tools like MS-Paint or GIMP to create the masks.

5. Define the prompt and pass the image, mask, and the prompt through the pipeline:

```
prompt = "Face of a white cat, high resolution, sitting on a park bench"  
image = pipeline(prompt=prompt, image=init_image,  
                 mask_image=mask_image).images[0]
```

Now, we can generate the image that corresponds to the prompt as well as the input image.



Figure 17.4: The in-painted image

In this section, we learned about replacing the subject of an image with another subject of our choice. In the next section, we'll learn about having the generated image in a certain pose of interest.

ControlNet

Imagine a scenario where we want the subject of an image to have a certain pose that we prescribe it to have – ControlNet helps us to achieve that. In this section, we will learn about how to leverage a diffusion model and modify the architecture of ControlNet and achieve this objective.

Architecture

ControlNet works as follows:

1. We take human images and pass them through the OpenPose model to get stick figures (keypoints) corresponding to the image. The OpenPose model is a pose detector that is very similar to the human pose detection model that we explored in *Chapter 10*.
 1. The inputs to the model are a stick figure and a prompt corresponding to the image, and the expected output is the original human image.
2. We create a replica of the downsampling blocks of the UNet2DConditionModel (the copies of the downsampling blocks are shown in *Figure 17.5*).
3. The replica blocks are passed through a zero-convolution layer (a layer with the weight initialization set to zero). This is done so that we can train the model faster. If they were not passed through zero-convolution layer, the addition of the replica blocks could modify the inputs (which include the text latents, the latents of the noisy image, and the latents of the input stick figure) to the upsampling blocks, resulting in a distribution that the upsamplers have not seen before (for example, facial attributes in the input image are preserved when the replica blocks do not contribute much initially).
4. The output of the replica blocks is then added to the output from the original downsampling blocks while performing upsampling.
5. The original blocks are frozen, and only the replica blocks are set to train.
6. The model is trained to predict the output (in a given pose, that of the stick figure) when the prompt and stick figure are the inputs.

This workflow is illustrated in the following diagram:

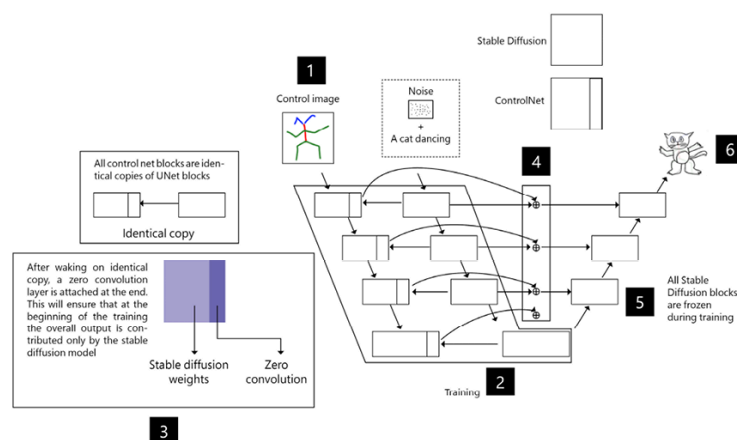


Figure 17.5: ControlNet workflow

Note that the exact same pipeline can be extended to not only canny images but also rough lines, scribbles, image segmentation maps, and depth maps.

Now that we understand the way in which ControlNet is trained, let us go ahead and code it up in the next section.

Implementing ControlNet

To implement ControlNet, we will leverage the `diffusers` library and a pre-trained model that is trained to predict an image given an image and prompt. Let us code it up:



The following code is available in the `ControlNet_inference.ipynb` file of the `Chapter17` folder in the GitHub repository at <https://bit.ly/mcvp-2e>.

1. Install the required libraries and import them:

```
%pip install -Uqq torch-snippets diffusers accelerate
from torch_snippets import *
import cv2
import numpy as np
from PIL import Image
```

2. Extract a canny edge image from a given image:

```
image0 = read('/content/Screenshot 2023-11-06 at 11.14.37 AM.png')
low_threshold = 10
high_threshold = 250
image = cv2.Canny(image0, low_threshold, high_threshold)
image = image[:, :, None]
canny_image = np.concatenate([image, image, image], axis=2)
canny_image = Image.fromarray(canny_image)
show(canny_image, sz=3)
canny_image.save('canny.png')
```

The preceding code results in a canny image, as follows:

```
subplots([image0, canny_image])
```

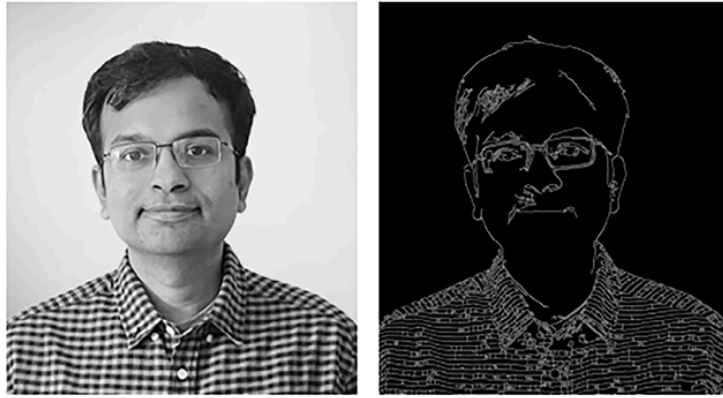


Figure 17.6: The original image (left) and the canny image (right)

3. Import the modules that help in implementing ControlNet from the `diffusers` library:

```
from diffusers import StableDiffusionControlNetPipeline, ControlNetModel, UniPCMultistepScheduler
from diffusers.utils import load_image
import torch
```

4. Initialize ControlNet:

```
generator = torch.Generator(device='cuda').manual_seed(12345)
controlnet = ControlNetModel.from_pretrained(
    "lllyasviel/sd-controlnet-canny")
```

In the preceding code, we load the pretrained `ControlNet` model.

5. Define the pipeline and noise scheduler to generate an image:

```
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    controlnet=controlnet
).to('cuda')
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
```

The architecture of the pipeline defined above is provided in the GitHub notebook. The architecture contains the different models used to extract encoders from the input image and prompt.

6. Pass the canny image through the pipeline:

```
image = pipe(
    "a man with beard, realistic, high quality",
    negative_prompt="cropped, out of frame, worst quality, low quality, jpeg artifacts, ugly, l",
    num_inference_steps=100,
    generator=generator,
    image=canny_image,
    controlnet_conditioning_scale=0.5
```

```
) .images[0]  
image.save('output.png')
```

The preceding code results in the following output:



Figure 17.7: The output image

Note that the generated image is very different from the person that was originally there in the image. However, while a new image is generated as per the prompt, the pose that is present in the original image is preserved in the generated image. In the next section, we will learn how to generate high-quality images quickly.

SDXL Turbo

Much like Stable Diffusion, a model called **SDXL (Stable Diffusion Extra Large)** has been trained that returns HD images that have dimensions of 1,024x1,024 . Due to its large size, as well as the number of denoising steps, SDXL takes considerable time to generate images over increasing time steps. How do we reduce the time it takes to generate images while maintaining the consistency of images? SDXL Turbo comes to the rescue here.

Architecture

SDXL Turbo is trained by performing the following steps:

1. Sample an image and the corresponding text from a pre-trained dataset (the **Large-scale Artificial Intelligence Open Network (LAION)** available at <https://laion.ai/blog/laion-400-open-dataset/>).
2. Add noise to the original image (the chosen time step can be a random number between 1 and 1,000)
3. Train the student model (the Adversarial diffusion model) to generate images that can fool a discriminator.
4. Further, train the student model in such a way that the output is very similar to the output of the teacher SDXL model (when the noise-added output from the student model is passed as input to the teacher

model). This way, we optimize for two losses – discriminator loss (between the image generated from the student model and the original image) and MSE loss between the outputs of the student and teacher models. Note that we are training the student model only.

This is illustrated in the following diagram:

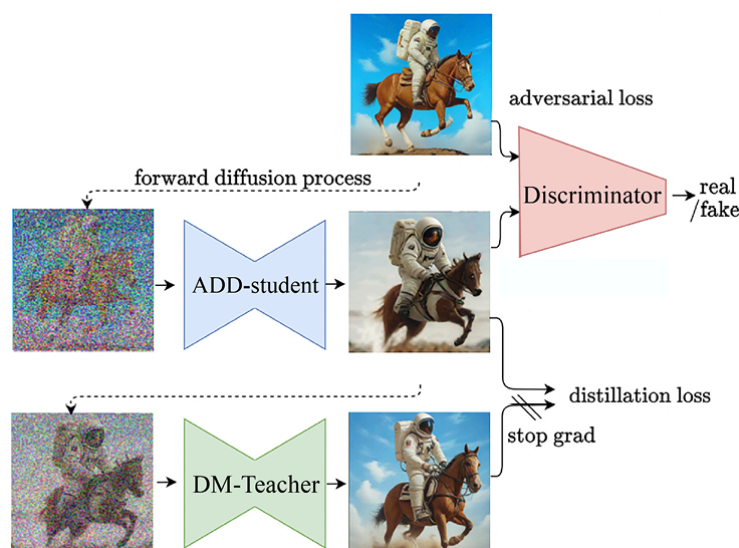


Figure 17.8: SDXL turbo training

Source: <https://stability.ai/research/adversarial-diffusion-distillation>

Training for both adversarial loss and distillation loss could help the model to generalize well even for minor modifications to the input image (the output of the teacher model).

Implementing SDXL Turbo

SDXL Turbo is implemented in code as follows:

You can find the code in the `sdxl_turbo.ipynb` file in the Chapter17 folder of the GitHub repository at <https://bit.ly/mcnp-2e>.

1. Install the required libraries:

```
%pip -q install diffusers accelerate torch-snippets torchinfo lovely_tensors
```

2. Import the required packages:

```
from diffusers import AutoPipelineForText2Image, AutoPipelineForImage2Image
import torch
```

3. Define the `sdxl-turbo` pipeline:

```
pipeline = AutoPipelineForText2Image.from_pretrained("stabilityai/sdxl-turbo", torch_dtype=torch.float16, variant="fp16")
pipeline = pipeline.to("cuda")
```

4. Provide the prompt and the negative prompt (`n_prompt`) and fetch the output image. Note that the negative prompt (`n_prompt`) ensures that the attributes mentioned in it are not present in the generated image:

```
%%time
prompt = "baby in superman dress, photorealistic, cinematic"
n_prompt = 'bad, ugly, blur, deformed'
image = pipeline(prompt, num_inference_steps = 1,
                 guidance_scale=0.0, negative_prompt = n_prompt,
                 seed = 42).images[0]

image
```



Figure 17.9: The generated image

The preceding code is executed in less than 2 seconds, while a typical SDXL model takes more than 40 seconds to generate one image.

DepthNet

Imagine a scenario where you want to modify the background while keeping the subject of the image consistent. How would you go about solving this problem?

One way to do this is by leveraging the **Segment Anything Model (SAM)**, which we learned about in *Chapter 16*, and replacing the background with the background of your choice. However, there are two major problems associated with this method:

- The background is not generated, and so you will have to manually provide the background image.
- The subject and background will not be color-consistent with each other because we have done patchwork.

DepthNet solves this problem by leveraging a diffusion approach, where we will use the model to understand which parts of an image are the

background and foreground using a depth map.

Workflow

DepthNet works as follows:

1. We calculate the depth mask of an image (depth is calculated by leveraging a pipeline similar to the one mentioned in the *Vision Transformers for Dense Prediction* paper: <https://arxiv.org/abs/2103.13413>).
2. The diffusion UNet2DConditionModel is modified to accept a five-channel input, where the first four channels are the standard noisy latents and the fifth channel is simply the latent depth mask.
3. Now, train the model to predict the output image using the modified diffusion model, where, along with a prompt, we also have an additional depth map as input.

A typical image and its corresponding depth map are as follows:

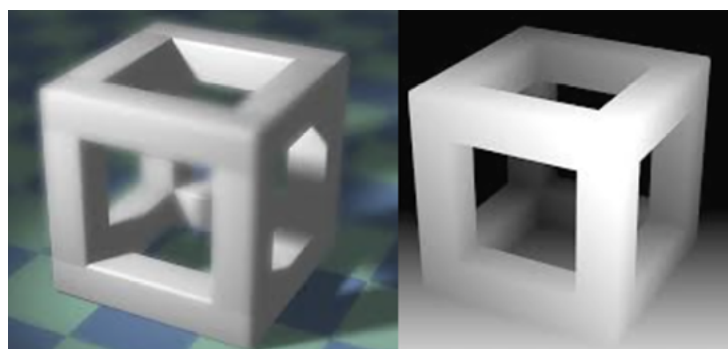


Figure 17.10: An image and its depth map

Let's now go ahead and implement DepthNet.

Implementing DepthNet

To implement DepthNet, you can use the following code:



The full code can be found in the `DepthNet.ipynb` file of the Chapter17 folder in the GitHub repository at <https://bit.ly/mcnp-2e>.

1. Install the required packages:

```
%pip -q install diffusers accelerate torch-snippets torchinfo lovely_tensors
!wget https://png.pngtree.com/thumb_back/fw800/background/20230811/pngtree-two-glasses-with-a-
```

2. Import the required packages:

```
import torch
import requests
from PIL import Image
```

```
from torch_snippets import *
from diffusers import StableDiffusionDepth2ImgPipeline
```

3. Define the pipeline:

```
pipe = StableDiffusionDepth2ImgPipeline.from_pretrained(
    "stabilityai/stable-diffusion-2-depth",
    torch_dtype=torch.float16,)
```

4. Specify the prompt and pass the image through the pipeline:

```
init_image = Image.open('/content/test.jpg')
prompt = "glasses of lime juice with skyline view in background in a cool afternoon"
n_prompt = "bad, deformed, ugly, bad anatomy"
image = pipe(prompt=prompt, image=init_image,
             negative_prompt=n_prompt, strength=0.7,
             num_inference_steps = 100).images[0]
```

The preceding code results in the following output (for the colored images, you can refer to the digital version of the book):



Figure 17.11: (Left) The input image (Right) The output from DepthNet

Note that in the above picture, the depth in the original picture (the picture on the left) is maintained while the prompt modified the content/view of the image.

Text to video

Imagine a scenario where you provide a text prompt and expect to generate a video from it. How do you implement this?

So far, we have generated images from a text prompt. Generating videos from text requires us to control two aspects:

- **Temporal consistency** across frames (the subject in one frame should look similar to the subject in a subsequent frame)
- **Action consistency** across frames (if the text prompt is a rocket shooting into the sky, the rocket should have a consistent upward trajectory over increasing frames)

We should address the above two aspects while training a text-to-video model, and the way we address these aspects again uses diffusion models.

To understand the model building process, we will learn about the text-to-video model built by damo-vilab. It leverages the

`Unet3DConditionModel` instead of the `Unet2DConditionModel` that we saw in the previous chapter.

Workflow

The `Unet3DConditionModel` contains the `CrossAttnDownBlock3D` block instead of the `CrossAttnDownBlock2D` block. In the

`CrossAttnDownBlock3D` block, there are two modules in addition to the `resnet` and `attention` modules that we saw in the previous chapter:

1. **temp_conv**: In the `temp_conv` module, we pass the inputs through a `Conv3D` layer. The inputs in this case take all the frames into account (while in 2D, it was one frame at a time). In essence, by considering all the frames together, our input is a 5D tensor with the shape [bs, frames, channels, height, width]. You can consider this as a mechanism to maintain the temporal consistency of a subject across frames.
2. **temp_attn**: In the `temp_attn` module, we perform self-attention on the frame dimension instead of the channel dimension. This helps to maintain action consistency across frames.

The `CrossAttnUpBlock3D` and `CrossAttnMidBlock3D` blocks differ only in their submodules (which we have already discussed above) and have no functional differences compared to their 2D counterparts. We will leave gaining an in-depth understanding of these blocks as an activity for you.

Implementing text to video

Let's now go ahead and implement the code to perform text-to-video generation:



The following code is available in the `text_image_to_video.ipynb` file of the `Chapter17` folder in the GitHub repository at <https://bit.ly/mcvp-2e>.

1. Install the required packages and import them:

```
%pip -q install -U diffusers transformers accelerate torch-snippets lovely_tensors torchinfo py
import torch
from diffusers import DiffusionPipeline, DPMSolverMultistepScheduler
from diffusers.utils import export_to_video
from IPython.display import HTML
from base64 import b64encode
```

2. Define the pipeline for text-to-video generation:

```
pipe = DiffusionPipeline.from_pretrained(\
    "damo-vilab/text-to-video-ms-1.7b",\
    torch_dtype=torch.float16, variant="fp16")
pipe.enable_model_cpu_offload()
```

3. Provide the prompt, video duration, and number of frames per second to generate the video:

```
prompt = 'bear running on the ocean'
negative_prompt = 'low quality'
video_duration_seconds = 3
num_frames = video_duration_seconds * 25 + 1
```

4. Pass the above parameters to the pipeline:

```
video_frames = pipe(prompt, negative_prompt="low quality", num_inference_steps=25, num_frames=1
```

5. Display the video using the following code:

```
def display_video(video):
    fig = plt.figure(figsize=(4.2,4.2)) #Display size specification
    fig.subplots_adjust(left=0, right=1, bottom=0, top=1)
    mov = []
    for i in range(len(video)): #Append videos one by one to mov
        img = plt.imshow(video[i], animated=True)
        plt.axis('off')
        mov.append([img])
    #Animation creation
    anime = animation.ArtistAnimation(fig, mov,
                                     interval=100, repeat_delay=1000)

    plt.close()
    return anime
video_path = export_to_video(video_frames)
video = imageio.mimread(video_path) #Loading video
HTML(display_video(video).to_html5_video()) #Inline video display in HTML5
```

With the above, we can now generate video from text. You can take a look at the generated video in the associated notebook.

Summary

In this chapter, we learned about creative ways to leverage a diffusion model for multiple applications. In the process, we also learned about the working details of various architectures along with the code implementations.

This, in conjunction with the strong foundations of understanding how diffusion models work, will ensure that you are able to leverage Stable Diffusion models for multiple creative works, modify and fine-tune architectures for custom image generation, and combine/pipeline multiple models to get the output you're looking for.

In the next chapter, we will learn about deploying computer vision models and the various aspects that you need to consider when doing so.

Questions

1. What is the key concept behind image in-painting using Stable Diffusion?

2. What are the key concepts behind ControlNet?
3. What makes SDXL Turbo faster than SDXL?
4. What is the key concept behind DepthNet?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

